Connecting the Cognitum.One Seed's MCP Server to Claude

Connecting the Cognitum.One Seed's MCP Server to Claude

By Mondweep Chakravorty · GitHub

Executive Summary

Goal. Make the Cognitum.One USB seed's MCP server visible inside a Claude session — both Claude Desktop / Cowork and Claude Code CLI — so Claude can call seed tools directly (`seed.device.status`, `seed.memory.query`, `seed.witness.chain`, etc.).

The short version of why this was hard. The Cognitum seed runs MCP protocol version `2025-11-25`, which is *newer than what shipping Claude clients support*. Both Claude Desktop and Claude Code use MCP SDKs whose Zod schema validators reject several fields the seed sends (`tasks` capability, per-tool `execution`, non-standard tool annotations). The rejection is silent — the connection appears to succeed but no tools are exposed.

What we ended up with. A custom 70-line Node.js stdio↔HTTP bridge (`cognitum-bridge.mjs`) that:

1. Talks streamable HTTP to the seed at `http://169.254.42.1/mcp` over USB Ethernet.
2. Injects the bearer token from pairing.
3. Strips fields that Claude's MCP SDK doesn't recognize from `initialize` and `tools/list` responses before passing them through.

Registered with Claude Code at user scope:

```
claude mcp add cognitum-seed -s user -- node
/Users/mondweep/Documents/Claude/Projects/Cognitum.One/cognitum-bridge.mjs
```

Result: ~40 seed tools across 12 groups visible in any Claude Code session.

Things that did NOT work, in case anyone tries them again:

- Claude Desktop's `claude_desktop_config.json` with `type: http` or `transport: streamable-http`. Silently rejected as "invalid MCP server config entries" — only stdio is accepted in that file.

- Claude Desktop's "Custom Connectors" UI. Requires HTTPS + OAuth, no field for static bearer tokens.
- The `npx mcp-remote` shim. Rejects the seed's `initialize` response with a Zod schema error on `capabilities.tasks`.
- Claude Code's native `--transport http`. Same Zod rejection, silent — `claude mcp list` just says "Failed to connect" with no detail.
- Trusting the seed's CA via macOS Keychain so HTTPS validates. Even after `security add-trusted-cert -d -r trustRoot -p ssl ...` and `security verify-cert` returning "successful", Safari and `/usr/bin/curl` still rejected. Cert chain itself is fine; trust just doesn't propagate to TLS validators on this macOS version.
- The `@cognitum/cog-dev` npm package referenced in the seed's guide. Not actually published.

The takeaway: the seed is on a newer MCP spec than current clients. Until clients catch up, you need a translation shim. Ours does only the minimum.

Bugs to Report Upstream

Each subsection below is designed to be copy-pasted directly into a GitHub issue against the relevant project — title is the section header, body is the rest. Severity and triage hints are at the top of each.

#	Title	Severity	Component
1	MCP `initialize` and `tools/list` responses contain non-standard schema fields that break standard MCP clients	**High**	seed firmware (MCP server)
2	Documented `@cognitum/cog-dev` npm package is not published on the public registry	Medium	seed firmware (docs/getting-started)
3	`POST /api/v1/store/truncate-confirm` returns `EPERM` on the data file the agent has open	Medium	seed firmware (recovery API)
4	Seed boot guide URL needs HTTP-first sequence to provision	Low	seed firmware (docs)

#	Title	Severity	Component
	HTTPS — tripping wire for new users		

Bug 1 — MCP `initialize` and `tools/list` responses contain non-standard schema fields that break standard MCP clients

Severity: High. Silently breaks every shipping MCP client we tried, with no indicative error message.
Affected clients: Claude Desktop (current), Claude Code 2.1.131, `mcp-remote` (latest @ npm), and presumably anything else built on `@modelcontextprotocol/sdk` that hasn't shipped support for protocol revision `2025-11-25`. **Firmware observed:** `cognitum-seed v0.21.11`. Server reports `protocolVersion: "2025-11-25"`.

Reproduction

1. Pair the seed and obtain a bearer token.
2. From a Mac with Claude Code installed, run:

```
claude mcp add cognitum-seed --transport http \
  --header "Authorization: Bearer <token>" \
  http://<seed-ip>/mcp
claude mcp list
```

3. Observed: `cognitum-seed: ... ✗ Failed to connect`. No detail in `claude --debug` output.

The same symptoms surface using `mcp-remote`, which does expose the underlying error:

```
$ npx -y mcp-remote http://<seed-ip>/mcp --allow-http \
    --header "Authorization: Bearer <token>"
[xxxx] Connection error: ZodError: [
  { "code":"custom", "path":["capabilities","tasks","list"],   "message":"Invalid input" },
  { "code":"custom", "path":["capabilities","tasks","cancel"], "message":"Invalid input" }
]
```

A second, distinct rejection appears at `tools/list` time: in Claude Desktop the MCP panel shows `cognitum-seed · △ connected · no tools`, and in `mcp-remote` (with `--debug`) the per-tool schema validation rejects fields not in the standard tool spec.

Root cause

Two non-standard schema additions:

1. **`initialize` response**: `result.capabilities.tasks: { cancel: true, get: true, list: true }`. The MCP SDKs' Zod schemas reject the `tasks` capability key. (This is part of the new

server-side async tasks feature in MCP `2025-11-25`, but the rest of the SDK ecosystem hasn't shipped support.)

2. **`tools/list` response, every tool**: each tool descriptor contains:
   - `execution: { taskSupport: "none" | "optional" }` — non-standard top-level field.
   - `annotations.{authClass, scope, group, relatedTools}` — non-standard annotation keys. Standard MCP annotations are limited to `title`, `readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`.

Both rejections are silent: the MCP SDK's Zod `.strict()` schemas drop the response and the client either reports "Failed to connect" or "connected · 0 tools" with no diagnostic.

Suggested fix (any one would unblock current clients)

1. **Inspect `clientInfo.protocolVersion` from the `initialize` request** and only emit `tasks` capability + per-tool `execution` for clients whose declared version is `>= 2025-11-25`. For older clients, omit those fields and downgrade to `protocolVersion: 2024-11-05` (or whatever the highest version both sides support).
2. **Move the non-standard tool annotations** (`authClass`, `scope`, `group`, `relatedTools`) under `annotations.experimental` — the spec is permissive about what lives there.
3. **At minimum, document the requirement** to use a sanitizing shim, with a reference implementation. We have one we can share (~70 lines of Node.js).

Workaround

Custom Node.js stdio↔HTTP bridge that strips the offending fields from the seed's responses before passing them to the MCP client. Source: see `cognitum-bridge.mjs` in this repo. The interesting bit is the `sanitizePayload()` function — ~15 lines.

Bug 2 — Documented `@cognitum/cog-dev` npm package is not published on the public registry

Severity: Medium. Breaks the documented onboarding command. New users will hit this immediately. **Affected docs:** Both the bundled local guide at `http://<seed-ip>/guide` and the public guide at `https://seed.cognitum.one/guide.html`. **Firmware observed:** `cognitum-seed v0.21.11`.

Reproduction

The guide's "Quick connect" section instructs:

> `npx -y @cognitum/cog-dev mcp connect --auto-discover` Wraps `claude mcp add` with seed auto-discovery and bundled MCP transport.

Run that command:

```
$ npx -y @cognitum/cog-dev mcp connect --auto-discover
npm error code E404
npm error 404 Not Found - GET https://registry.npmjs.org/@cognitum%2fcog-dev - Not found
npm error 404
npm error 404  '@cognitum/cog-dev@*' is not in this registry.
```

Suggested fix

Either publish `@cognitum/cog-dev` to the public npm registry, or update both guide pages to remove the reference and provide the manual `claude mcp add` recipe instead. (Manual setup also requires the workaround in Bug 1, so any "official" wrapper would need to either include the sanitizing shim or wait for upstream MCP SDKs to catch up to `2025-11-25`.)

Bug 3 — `POST /api/v1/store/truncate-confirm` returns `EPERM` when called against a real corrupted store

Severity: Medium. The documented recovery path for witness-chain corruption doesn't work, leaving operators no choice but a manual SSH wipe. **Firmware observed:** `cognitum-seed v0.21.11`. **Trigger:** Seed had been upgraded from a pre-v0.21.11 firmware version where the witness-chain corruption bug exists. On boot, the agent detects the bad genesis entry and self-truncates with `writes_gated: true`, instructing the operator to call `truncate-confirm` to acknowledge.

Reproduction

State of the seed before:

```
"integrity": {
  "truncated_at_entry": 0,
  "truncated_at_offset": 58043,
  "truncation_reason": "witness chain corrupted at entry id=0; truncated here to allow agent to
        boot. Writes are gated until operator confirms via POST /api/v1/store/truncate-confirm.
        To preserve evidence: back up /var/lib/cognitum/rvf-store/memopt.rvf before confirming.",
  "writes_gated": true,
  ...
}
```

Call the endpoint as the docs instruct:

```
$ curl -sk -X POST https://<seed-ip>:8443/api/v1/store/truncate-confirm \
    -H "Authorization: Bearer <token>"
{"error":"truncate set_len(58043): Operation not permitted (os error 1)"}
```

`set_len(...)` corresponds to the Rust `std::fs::File::set_len` → Linux `ftruncate(2)`. `EPERM` here is unusual — the more common errors are `EBUSY`, `EACCES`, or success-but-mmap-zaps. Possible underlying causes:

- The cognitum-agent process is holding the file with an active mmap that prevents shrinking past the mapped region.
- `chattr +i` (immutable attribute) on the file. (Confirmed *not* the case here — `lsattr` returned no flags.)
- Filesystem is read-only. (Not the case — verified rw via `df -T` and other writes work.)

Most plausible: the agent has the file open with an `mmap(MAP_SHARED)` covering the active witness chain region, and the kernel rejects `ftruncate` because shrinking would invalidate the mapping. The endpoint handler should drop the mapping (or close+reopen the file handle) before truncating.

Workaround

Manual wipe via SSH — works but loses any data the operator might have wanted to preserve through the truncation point:

```
ssh genesis@<seed-ip>          # default password: cognitum
sudo systemctl stop cognitum-agent
sudo rm /var/lib/cognitum/rvf-store/memopt.rvf
sudo systemctl start cognitum-agent
exit
```

After this, `curl http://<seed-ip>/api/v1/status` returns `writes_gated: false`, `truncation_reason: null`, `total_vectors: 0`, `witness_chain_length: 0` — clean genesis. The seed begins accumulating new witness entries from its own cognitive container ticks within seconds.

Suggested fix

Inside the `truncate-confirm` endpoint handler:

1. Drop any active mmap of `memopt.rvf` before calling `set_len`.
2. Or use the safer atomic-replace pattern: write the truncated content to `memopt.rvf.tmp`, `rename(2)` over the original, then re-open.
3. Or stop & restart the writer half of the agent's state machine internally rather than mutating an open fd.

Note on the witness-chain corruption bug itself

Per Cognitum support, the underlying corruption was a known bug in pre-v0.21.11 firmware. v0.21.11 and v0.21.12 are reported to fix the bug forward — once the corrupted state is cleared, new writes should not recurrence it. This bug report is specifically about the *recovery path* not working, not the corruption itself.

Bug 4 — First-boot HTTPS guide URL is unreachable; documentation should lead with the HTTP URL

Severity: Low. Causes confusion for first-time users; not a blocker but a needless friction point.
Firmware observed: `cognitum-seed v0.21.11`.

Reproduction

A first-time user follows printed/stickered guidance pointing at `https://<seed-ip>:8443/guide` and gets a connection failure. The seed's HTTPS endpoint (port 8443) is only active *after* WiFi setup → cert auto-provision (~2 min) → seed reboot. On first boot, only HTTP (port 80) is up, so `http://<seed-ip>/guide` works but `https://<seed-ip>:8443/guide` does not.

The current guide does mention this in the Quick Start text, but the seed's own onboarding stickers / external docs / `seed.cognitum.one` don't make it the obvious *first* URL to try.

Suggested fix

Use `http://<seed-ip>/guide` as the canonical first-touch URL in all onboarding materials. The HTTPS URL becomes useful after step 3 of the Quick Start, not before.

Out of scope for upstream issues

Two items came up during debugging that are *not* Cognitum bugs:

- **Claude Desktop's "Custom Connectors" UI requires HTTPS + OAuth.** No field for static bearer tokens. This is a Claude Desktop product limitation and not actionable on the seed side. (If Cognitum wants their seed to be addable from that UI, they'd need to implement OAuth — a much larger ask.)
- **macOS Keychain `add-trusted-cert` did not propagate trust to Safari/ `curl`** despite `verify-cert` returning success. Probably a macOS Sequoia/Tahoe quirk in how trust settings are written via the `security` CLI. The seed's CA cert chain itself is fine (subject `CN=Cognitum Device CA` self-signed, valid 2026–2046, `CA:TRUE`, `Certificate Sign + CRL Sign` key usage; server cert SANs include `IP:169.254.42.1`, `127.0.0.1`, and the `.local` hostnames). Workaround: bridge over plain HTTP via USB sidesteps this entirely.

Goal

Get Claude to call the seed's MCP tools so it can read device status, query the vector store, inspect the witness chain, manage cogs, etc., without the human manually `curl`ing the API.

Environment

- Mac, macOS Sequoia/Tahoe-era (2026).

- Cognitum.One USB seed device, firmware 0.21.11, paired, in `writes_gated:true` mode (witness-chain truncation pending operator confirmation — separate issue).
- Connected via USB. The seed presents:
  - A `COGNITUM` USB drive (storage).
  - A USB Ethernet adapter providing link-local network at `169.254.42.1`.
  - Optionally mDNS at `cognitum.local` (didn't resolve on this Mac — used IP throughout).
- Mac side of the USB Ethernet link: visible in System Settings → Network as "Cognitum Seed", self-assigned IP `169.254.30.244`.
- Plain HTTP MCP endpoint: `http://169.254.42.1/mcp` (port 80, always available).
- HTTPS endpoint: `https://169.254.42.1:8443/mcp` (port 8443, only after WiFi setup → cert auto-provision → seed reboot).
- Bearer token from pairing: `<REDACTED-bearer-token>`.

Sanity Check (the "it should just work" test)

Plain HTTP MCP via curl works first time and confirms the seed is healthy:

```
curl -s -X POST http://169.254.42.1/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize",
       "params":{"protocolVersion":"2025-11-25",
                 "clientInfo":{"name":"sanity","version":"1.0"},
                 "capabilities":{}}}'
```

Returns `{"jsonrpc":"2.0","result":{"capabilities":{...},"protocolVersion":"2025-11-25","serverInfo":{"name":"cognitum-seed","version":"0.21.11"}},"id":1}`.

So the seed itself is fine. Everything past this point is about the *client side* — Claude — being able to consume what the seed sends.

What We Tried — and Why Each Failed

Attempt 1 — Claude Desktop config file with HTTP transport

```
// ~/Library/Application Support/Claude/claude_desktop_config.json
"cognitum-seed": {
  "url": "http://169.254.42.1/mcp",
  "transport": "streamable-http",
  "headers": { "Authorization": "Bearer ..." }
}
```

Result. Silent rejection. `~/Library/Logs/Claude/main.log` showed:

```
[warn] Skipped invalid MCP server config entries: { invalidServers: [ 'cognitum-seed' ] }
```

Cause. This Claude Desktop version's config validator only accepts stdio entries (`command + args + type: stdio`). Switching `transport` to `type: "http"` didn't help. Remote MCPs in Claude Desktop go through a different system (Connectors UI, see Attempt 2), not this file.

Attempt 2 — Claude Desktop "Custom Connectors" UI

URL: `https://169.254.42.1:8443/mcp`. UI showed "Connect" button, no auth-header field.

Result. "Connect" opens a browser window for OAuth. Connection registers in your account but always fails — no OAuth flow can complete because the seed doesn't speak OAuth.

Cause. Claude Desktop's Custom Connectors path is OAuth-only. There's no place to paste a static bearer token. Two side-effects: (a) HTTPS-only restriction blocks the plain HTTP endpoint, (b) the broken entry syncs through your account into `claude mcp list` and keeps showing "Failed to connect" forever.

Attempt 3 — `mcp-remote` npm shim

Tried wrapping the HTTP endpoint with the standard community shim:

```
"cognitum-seed": {
  "command": "npx",
  "args": ["-y", "mcp-remote", "http://169.254.42.1/mcp",
           "--allow-http", "--transport", "http-only",
           "--header", "Authorization: Bearer ..."],
  "type": "stdio"
}
```

Result.

```
Connection error: ZodError: [
  { "code":"custom", "path":["capabilities","tasks","list"], "message":"Invalid input" },
  { "code":"custom", "path":["capabilities","tasks","cancel"], "message":"Invalid input" }
]
```

Cause. The seed advertises a `tasks` capability (`{cancel:true, get:true, list:true}`) introduced in MCP `2025-11-25`. `mcp-remote`'s bundled MCP TypeScript SDK predates that version, so its Zod schema rejects the unknown keys.

Attempt 4 — HTTPS + macOS Keychain trust + Custom Connectors UI

The plan: get the seed's HTTPS endpoint trusted at the OS level so the Connectors UI accepts the URL.

- Rebooted seed, confirmed `curl -k https://169.254.42.1:8443/api/v1/status` worked.
- SSH'd in (default password `cognitum`), pulled `/var/lib/cognitum/tls/ca.pem`.
- Trusted via `sudo security add-trusted-cert -d -r trustRoot -p ssl -k /Library/Keychains/System.keychain ...`. Output: silent success.
- `security trust-settings-export` showed *no* Cognitum entry recorded despite the trust command "succeeding".
- Re-added at user scope with `security add-trusted-cert -r trustRoot -p ssl -k ~/Library/Keychains/login.keychain-db ...`.
- `security verify-cert -p ssl ...` returned `certificate verification successful.`
- Safari **still** showed "This Connection Is Not Private". `/usr/bin/curl` still rejected with `self signed certificate in certificate chain`.

Cause. Couldn't conclusively determine why macOS trust didn't propagate to TLS validators on this macOS version. Cert chain is correct (subject SANs include `IP:169.254.42.1`, `cognitum-2c3c.local`, etc.; issuer is the locally-trusted `CN=Cognitum Device CA` root with `CA:TRUE` and `Certificate Sign` key usage). Either a Keychain Access GUI step was needed (the GUI itself is hidden in newer macOS — Spotlight can find it but the typical path through `/Applications/Utilities/` no longer exists), or there's a system-level constraint we didn't dig deep enough to find. We pivoted rather than continue down this path.

Even if Safari had accepted the cert, the Connectors UI would still have wanted OAuth, so this path was a dead-end regardless.

Attempt 5 — Claude Code with native `--transport http`

```
claude mcp add cognitum-seed --transport http \
  --header "Authorization: Bearer ..." \
  http://169.254.42.1/mcp
```

Result. `claude mcp list` shows `✗ Failed to connect`. `claude --debug mcp list` produces no error detail in stdout/stderr. Direct curl against the same URL works.

Cause. Same Zod schema rejection as `mcp-remote` — Claude Code's MCP SDK doesn't recognize the `tasks` capability either. Failure is silent.

Attempt 6 — Custom Node.js stdio↔HTTP bridge

This is the path that finally worked, but it took three iterations to find all the things Claude Code's SDK rejects.

Iteration A: bare bridge

Just read JSON-RPC from stdin, POST to seed, write response to stdout.

Result. `claude mcp list` says ✗ Failed. Adding file logging revealed the bridge *was* being spawned and the handshake was completing. Claude Code was just rejecting the response.

Lesson. Claude Code's MCP SDK has the same `tasks` capability blind spot as `mcp-remote`. We need to strip it before passing the `initialize` response on.

Iteration B: bridge that strips unknown capabilities from `initialize` response

Added a `sanitizePayload` step that deletes any unknown top-level key from `result.capabilities`.

Result. `claude mcp list` reported `✓ Connected` for the first time. But `/mcp` inside an interactive Claude Code session showed:

```
cognitum-seed · △ connected · no tools
```

Despite the bridge log showing the seed had returned a `tools/list` result with all 40 tools.

Lesson. Capabilities passed validation, but the per-tool schema in the `tools/list` response still had unknown fields the SDK didn't like. The △ symbol meant "connection ok, tools rejected".

Iteration C: also strip per-tool `execution` and non-standard annotations

Each seed tool in the `tools/list` response carries:

```
{
  "name": "seed.device.status",
  "description": "...",
  "inputSchema": { ... },
  "annotations": {
    "authClass": "public",       // non-standard
    "scope": "default",          // non-standard
    "group": "device",           // non-standard
    "relatedTools": [...],       // non-standard
    "destructiveHint": false,    // standard
    "idempotentHint": true,      // standard
    "readOnlyHint": true         // standard
  },
  "execution": { "taskSupport": "none" }   // non-standard, related to `tasks` capability
}
```

Standard MCP tool annotations are only `title`, `readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`. The `execution` field is not in the spec at all — it's a vendor extension paired with the `tasks` capability we already had to remove.

Updated `sanitizePayload` to also walk `result.tools[*]` and: - Delete the `execution` field. - Strip non-standard keys from `annotations`.

Result. `/mcp` now reports `cognitum-seed · ✔ connected · 40 tools`. Calling tools works.

The Final Working Bridge

File: `/Users/mondweep/Documents/Claude/Projects/Cognitum.One/cognitum-bridge.mjs`

What it does, in order:

1. Reads JSON-RPC messages line-by-line from stdin (sent by Claude Code).
2. POSTs each to `http://169.254.42.1/mcp` with `Authorization: Bearer <token>`, `Content-Type: application/json`, and `Accept: application/json, text/event-stream`.
3. Maintains the `Mcp-Session-Id` returned by the seed and replays it on subsequent requests.
4. Treats HTTP 202 as "notification accepted, no body" — these are the seed's response to MCP notifications like `notifications/initialized`.
5. Parses HTTP 200 responses (plain JSON or SSE-framed `data: …` events).
6. **Sanitizes** before emitting:
   - In `initialize` results: strips top-level capability keys not in `{completion, logging, prompts, resources, tools, experimental, sampling, roots, elicitation}`.
   - In `tools/list` results: deletes `execution` and any non-standard keys under `annotations` from each tool.
7. Writes the cleaned response to stdout, one JSON line per message.
8. Tracks pending fetches so that when stdin closes (the test scenario), the process waits for in-flight requests before exiting rather than racing to die first.

Everything else passes through verbatim. Tool *calls* (`tools/call`) and resource reads (`resources/read`) hit the seed unchanged.

Optional debug logging is gated on `COGNITUM_DEBUG=1` env or `--debug` arg; structured operational events are unconditionally appended to `/tmp/cognitum-bridge.log` so we could watch what Claude Code sent vs. what came back from the seed.

Setup From Scratch — Reproducible Steps

For someone setting this up on another Mac:

1. **Plug in seed via USB data port.** Confirm `curl -s http://169.254.42.1/api/v1/status` returns JSON with `paired:true`.

2. **If not paired**, follow the seed's pairing flow at `http://169.254.42.1/guide` (HTTP only on first boot — HTTPS isn't active until WiFi setup + reboot). Save the bearer token; it's shown only once.

3. **Save the bridge** to a stable location, e.g. `~/Documents/Claude/Projects/Cognitum.One/cognitum-bridge.mjs`. Bake in your bearer token at the top, or set `COGNITUM_TOKEN` env var.

4. **Sanity-test the bridge:**

   ```
   echo '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-11-
           25","clientInfo":{"name":"t","version":"1.0"},"capabilities":{}}}' \
     | node /path/to/cognitum-bridge.mjs
   ```

   Expect a JSON line containing `serverInfo`.

5. **Register with Claude Code at user scope:**

   ```
   claude mcp add cognitum-seed -s user -- node /path/to/cognitum-bridge.mjs
   claude mcp list | grep cognitum   # expect ✓ Connected
   ```

   `-s user` is important — `-s local` (the default) ties the entry to a project root and Claude Code's interactive sessions don't pick it up reliably even when cwd matches.

6. **In any new terminal:**

   ```
   claude
   # then in the prompt:
   /mcp
   ```

   Should show `cognitum-seed · ✔ connected · 40 tools`. Ask the model "Call seed.device.status" — it'll invoke the tool and show the device JSON.

7. **(Optional) Clean up earlier broken entries.** If you registered a `Cognitum.One` HTTPS connector via Claude Desktop's Custom Connectors UI, remove it from Settings → Connectors → Cognitum.One → Remove. Otherwise it'll keep showing `Failed to connect` in `claude mcp list` indefinitely.

Recovery: Witness Chain Corruption (resolved 2026-05-06)

A previous Cognitum firmware version had a bug that corrupted the on-disk witness chain at entry id=0 (the genesis entry). Symptoms after upgrading the seed:

- `curl http://169.254.42.1/api/v1/status` returns `writes_gated: true`.
- `integrity.truncation_reason` says `"witness chain corrupted at entry id=0; truncated here to allow agent to boot. Writes are gated until operator confirms via POST /api/v1/store/truncate-confirm."`
- Read tools work fine. Any write fails with a gating error.

The seed is on v0.21.11/0.21.12, which fixes the bug going forward — once the corrupted state is cleared, new writes won't recurrence.

Path A — `truncate-confirm` API (the documented path)

```
curl -sk -X POST https://169.254.42.1:8443/api/v1/store/truncate-confirm \
  -H "Authorization: Bearer <your-token>"
```

In our case this returned:

```
{"error":"truncate set_len(58043): Operation not permitted (os error 1)"}
```

The agent had `/var/lib/cognitum/rvf-store/memopt.rvf` open in a way that prevented `ftruncate(2)` from shrinking it (likely because the file was held open by the running process or an mmap'd region was preventing the shrink). The error is Linux-level `EPERM`, not an MCP/HTTP issue. Worth pinging Cognitum support with that exact error string in case there's a tighter fix; in our session we pivoted to Path B.

Path B — Wipe and reset (the bulletproof path)

Stop the agent, delete the data file, restart the agent. The seed re-creates a clean genesis chain.

First, on your Mac shell:

```
ssh genesis@169.254.42.1
# At the password prompt, type: cognitum
```

Then, *only after* the prompt changes to `genesis@cognitum-2c3c:~ $`, run these one at a time:

```
sudo systemctl stop cognitum-agent
# password for sudo on the seed: cognitum (same as SSH)
sudo chattr -i /var/lib/cognitum/rvf-store/memopt.rvf 2>/dev/null   # in case of immutable bit
sudo rm -f /var/lib/cognitum/rvf-store/memopt.rvf
sudo systemctl start cognitum-agent
exit
```

Finally, back on your Mac, verify clean state:

```
curl -s http://169.254.42.1/api/v1/status \
  | python3 -m json.tool \
  | grep -E "writes_gated|epoch|witness_chain_length|total_vectors|truncation_reason"
```

Expected output: - `writes_gated: false` ✓ - `truncation_reason: null` ✓ - `epoch`, `total_vectors`, `witness_chain_length`: small non-zero numbers (the seed's cognitive container

ticks every 30 s and writes its own witness entries, so even a freshly-wiped seed accumulates a few quickly — that's normal).

To confirm writes work end-to-end:

```
curl -sk -X POST https://169.254.42.1:8443/api/v1/demo/ingest-sample \
  -H "Authorization: Bearer <your-token>"
# Expect:
        {"dimension":8,"ingested":100,"new_epoch":..,"rejected":0,"verified":..,"witness_head":"..."}
```

`rejected:0` means writes are flowing.

Two gotchas worth flagging

The paste-buffer trap. Don't paste a multi-line block (the SSH command + the commands meant to run inside SSH) at your Mac shell all at once. Zsh buffers the lines that come after `ssh ...` while SSH is waiting for the password, then runs them on your Mac after SSH closes. You'll get sudden `command not found: systemctl` errors and, more dangerously, macOS `sudo` password prompts that are actually asking for your **Mac login password** even though you're mentally still in "seed mode" and might type `cognitum`. Three wrong attempts and macOS sudo locks you out for a few minutes. Always either type lines after SSH is connected, or use `ssh user@host 'command1; command2'` to run a one-liner remotely.

Which password where. - `genesis@169.254.42.1's password:` → seed SSH, password is `cognitum`. - `[sudo] password for genesis:` while inside the SSH session → still `cognitum` (genesis user's sudo password). - `Password:` at your Mac shell after SSH has closed → your **Mac login password**.

Tell which environment you're in by the prompt: `genesis@cognitum-2c3c:~ $` is the seed, `(base) mondweep@MacBook-Pro-5 ~ %` is your Mac.

Other Open Items

- **HTTPS path still untrusted.** If you ever want to use the HTTPS endpoint (e.g. over WiFi instead of USB), the macOS Keychain trust we added should be redone via Keychain Access GUI rather than the `security` CLI, or via a `system.keychain` plist with explicit per-policy trust settings. We didn't push through this since the bridge over plain HTTP works fine over USB.
- **Cowork (Claude Desktop) integration.** This setup is Claude Code only. If you want the seed inside Cowork, point Cowork's `claude_desktop_config.json` at the same bridge as a stdio entry — the Custom Connectors UI route can't work for this device.

Files in This Project

- `cognitum-bridge.mjs` — the working bridge.
- `MCP_SETUP_NOTES.md` — this document.

Configuration Locations

- `~/.claude.json` — Claude Code's MCP server registry. The `cognitum-seed` entry lives here under user scope.
- `~/Library/Application Support/Claude/claude_desktop_config.json` — Claude Desktop's MCP config. Currently does *not* contain a working cognitum-seed entry; one was added during debugging and can be removed.
- `~/Library/Logs/Claude/main.log` — Claude Desktop logs. Useful for spotting "invalid MCP server config entries" rejections.
- `/tmp/cognitum-bridge.log` — bridge's operational log. Each line shows what was sent/received during the most recent runs.

Author

Mondweep Chakravorty - LinkedIn: linkedin.com/in/mondweepchakravorty - GitHub: github.com/mondweep

This guide and the accompanying `cognitum-bridge.mjs` were produced over the course of getting a Cognitum.One seed working with Claude Desktop (Cowork) and Claude Code on macOS in May 2026. The bug reports in the "Bugs to Report Upstream" section are documented based on direct reproduction. Feel free to fork, adapt, or reference — attribution appreciated but not required.

If this saved you time, ping me on LinkedIn or open an issue on GitHub. If your Cognitum seed is in a different state than mine was — different firmware, different host OS, different Claude version — I'd love to hear what worked or didn't.

Appendix A — Rotating the Cognitum.One Seed Bearer Token

Written 2026-05-06 after a leaked-token incident: the bearer token had been hardcoded as a fallback default in `cognitum-bridge.mjs` and inside `cognitum-esp32-v0.6.3/run_bridge.sh` and `wifi-csi-sensing-paper-public.md`, then pushed to a public GitHub repo before being noticed and scrubbed. History was rewritten and the files were cleaned, but the token had to be treated as compromised. This appendix captures the rotation procedure and impact analysis distilled from that incident.

A.1 How to rotate the token on the seed

This repo does not document a dedicated `/api/v1/auth/rotate` endpoint, and as of firmware `0.21.11` no such endpoint was found via casual probing. The two known mechanisms, in order of preference:

1. **Re-run the pairing flow** at `http://169.254.42.1/guide` (HTTP only on first boot — the HTTPS guide UI at `https://169.254.42.1:8443/guide.html` is broken; see §11.8 of the WiFi CSI sensing paper). Per the pairing flow's UI, *"the bearer token is shown only once"* — strongly implying re-pairing issues a fresh token. Whether the old token is automatically revoked or simply left valid is undocumented; assume not, and re-pair on a fresh device entry to be safe.
2. **Factory-reset the seed and re-pair from scratch.** Definitely invalidates the old token but also wipes seed state — the witness chain, the RVF vector store, and the paired-device list are all lost. Overkill if option 1 works. Use only if option 1 fails to invalidate the old token or the seed is in an unrecoverable state.

The seed itself ships an MCP tool group called `seed_guide_*` (visible in any Claude Code session once the bridge is connected). `seed_guide_explain` and `seed_guide_endpoints` are the authoritative reference for pairing/auth procedures on whatever firmware version is actually running. **When in doubt, ask the seed.**

A.2 Impact of rotation

Everything currently authenticating with the old token starts getting `401 Unauthorized` until updated.

Component	Where the token lives	Effect of rotation
`cognitum-bridge.mjs` (Claude Code → seed MCP)	`COGNITUM_TOKEN` env var	Claude Code's `seed.*` tools all fail until env var updated and bridge restarted.
`seed_csi_bridge.py` via `run_bridge.sh` (UDP CSI → seed RVF ingest)	`SEED_TOKEN` env var	CSI vector ingest stops; ESP32-sourced data accumulates in UDP socket buffer or is dropped.
Saved `claude mcp` registrations using `--header "Authorization: Bearer ..."`	Inside `~/.claude.json`	`claude mcp list` shows ✗. Re-register with the new token.
ESP32 nodes in **direct swarm mode (ADR-066)**	NVS `csi_cfg/seed_token`	ESP32s fail to ingest directly to seed. Re-provisioning required (see §A.3 Path B).

Not affected by rotation: the witness chain, the RVF store, the seed's TLS cert and `https://169.254.42.1:8443` endpoint identity, ESP32 WiFi credentials in NVS, ESP32 node IDs, channel/zone config.

A.3 What to do on the ESP32s

This depends on which architecture your nodes are using — the firmware supports two:

Path A — UDP-to-host (the mode `run_bridge.sh` runs):

```
ESP32 ──UDP/5006──► Mac (seed_csi_bridge.py) ──HTTPS Bearer──► Seed
```

The token lives only on the host. ESP32s know nothing about the seed token in this mode. **Action on ESP32: nothing.** Just `export SEED_TOKEN=<new>` on the host and restart `run_bridge.sh`.

Path B — Direct swarm bridge (ADR-066):

```
ESP32 ──HTTPS Bearer──► Seed
```

The token sits in ESP32 NVS under `csi_cfg/seed_token`, written by `provision.py --seed-token`. **Action on ESP32: re-provision each node.**

```
# IMPORTANT: provision.py REPLACES the entire csi_cfg NVS namespace (issue #391).
# You MUST pass all WiFi credentials again, or any unspecified key gets wiped.
python provision.py \
  --port /dev/tty.usbserial-XXXX \
  --ssid "<your-ssid>" \
  --password "<your-wifi-password>" \
  --target-ip "<host-ip>" \
  --seed-url "<seed-url>" \
  --seed-token "<new-token>" \
  # ... plus any tdm/edge/channel settings the node had
```

Read each node's existing config first (`esptool.py read_flash 0x9000 0x6000 nvs.bin` then inspect with `strings`) if you don't remember exact settings — otherwise you'll silently wipe them.

Quick test for which path you're on:

```
esptool.py --port /dev/tty.usbserial-XXXX read_flash 0x9000 0x6000 nvs.bin
strings nvs.bin | grep -E "seed_token|seed_url"
```

If `seed_token` appears in NVS, Path B is in use (or both). Otherwise just A.

A.4 Recommended order of operations

1. Decide Path A vs. B (see §A.3).

2. **Rotate token on the seed** — re-pair via `http://169.254.42.1/guide`, or query `seed_guide_explain` via the still-active MCP bridge to confirm the procedure for your firmware version.
3. **Update host env vars:** `COGNITUM_TOKEN` for Claude Code, `SEED_TOKEN` for the CSI bridge. Avoid embedding the new token in any committed file or in shell history (`HISTFILE=/dev/null` for the rotation session, or use a password manager and `read -rs`).
4. **Restart the Claude Code MCP bridge:** `claude mcp remove cognitum-seed && claude mcp add cognitum-seed -s user -- node /path/to/cognitum-bridge.mjs`. Verify with `claude mcp list`.
5. **Restart `run_bridge.sh`** (`./stop_bridge.sh` first, then `./run_bridge.sh`).
6. **If Path B**, re-provision each ESP32 with the new token plus the full WiFi trio.
7. **Verify CSI ingest is flowing again:** check `bridge.log` for fresh ingest lines and run `seed.rvf.query` from Claude Code.

A.5 Defensive practice — why the leak happened

The token landed in three files because it was convenient as a default during early bring-up:

- `cognitum-bridge.mjs:11` — `const TOKEN = process.env.COGNITUM_TOKEN || '<token>';` ("just so it works without setting the env var")
- `cognitum-esp32-v0.6.3/run_bridge.sh:13` — copy-paste-friendly help message inside an error branch.
- `wifi-csi-sensing-paper-public.md` §11.7 — example string in a discussion of bearer-token vs. pairing-code length.

In retrospect, none of the three needed the literal value. The fixes:

- The bridge now exits with a FATAL error if `COGNITUM_TOKEN` is unset (no silent fallback).
- The shell script's help message uses `<your-bearer-token>` placeholder.
- The paper redacts the literal value and only references the *length* of a 256-bit URL-safe-base64 token.

A second observation from the incident: **the running `seed_csi_bridge.py` process exposes the token in `argv`**, visible to any local user via `ps aux`. The python script accepts `--token` on the command line. For multi-user systems, prefer reading the token from an env var (already supported via `$SEED_TOKEN`) and pass `--token "$SEED_TOKEN"` only via env interpolation that's never inspected — or better, modify the script to read `$SEED_TOKEN` directly without a CLI flag at all. Not done in this repo; logged here as a follow-up for whoever picks this up next.